

Projet 16

EY & Associés – Forensic & Integrity

Création automatique de graphes de relations
à partir d'un rapport de mission



Sommaire

- Contexte / Objectif
- Développement
- Mise en commun
- Module de contrôle
- Limites et difficultés
- Méthodologie / Gantt
- Conclusion





Sommaire

- **Contexte / Objectif**
- Développement
- Mise en commun
- Module de contrôle
- Limites et difficultés
- Méthodologie / Gantt
- Conclusion





Contexte / Objectif

- EY : Cabinet d'**audit** et de **conseil financier**

Leurs missions :

- Enquêtes internes
- Vérifications d'antécédents
- Cartographier des réseaux d'influences

Processus :

- Lire les documents
- Extraire les données (noms des entités, relations etc...)
- Tracer à la main le graphe
 - Travail **chronophage**, **fastidieux** et sources d'**erreurs**

Notre mission : Automatiser ce processus





Sommaire

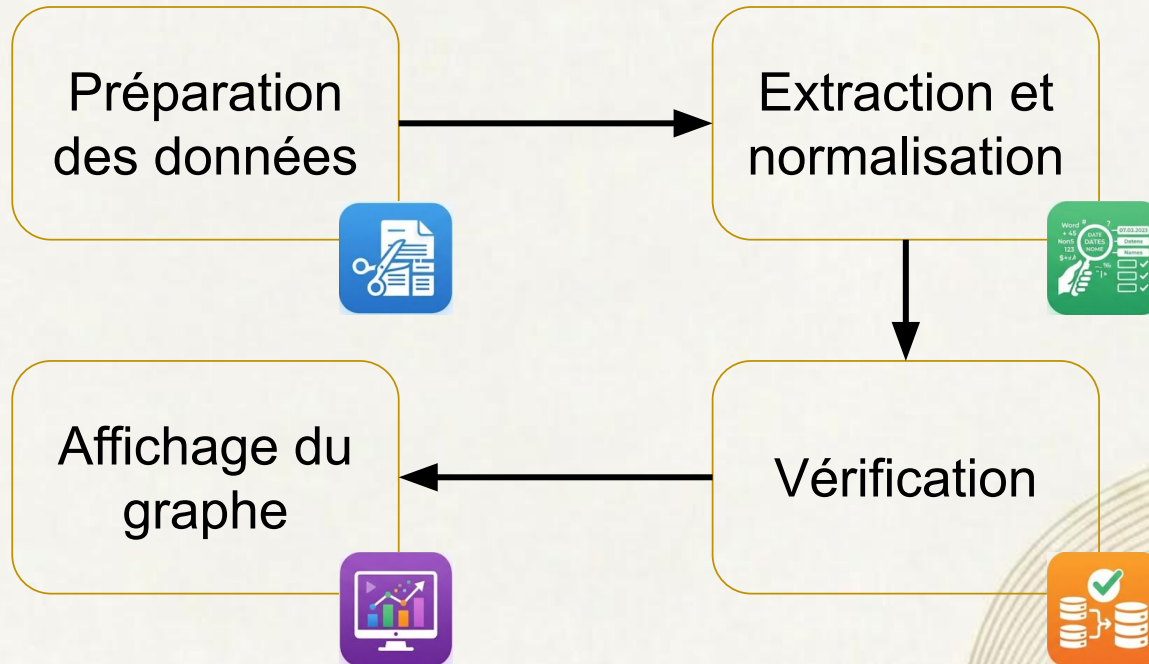
- Contexte / Objectif
- **Développement**
- Mise en commun
- Module de contrôle
- Limites et difficultés
- Méthodologie / Gantt
- Conclusion





Développement

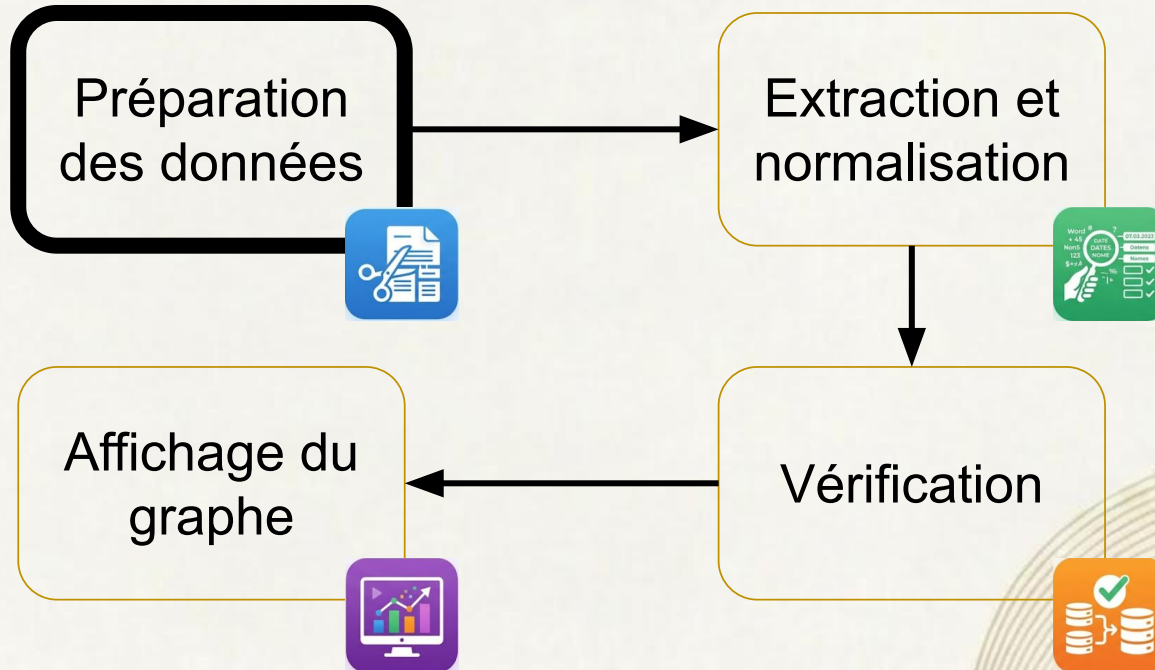
La partie de développement se découpe en **4 grandes étapes** :





Développement

La partie de développement se découpe en **4 grandes étapes** :





Préparation des données

Problèmes potentiels :

- Difficulté de traiter un texte entier
- Perte d'informations
- Diminution de la qualité des réponses
- Incohérences

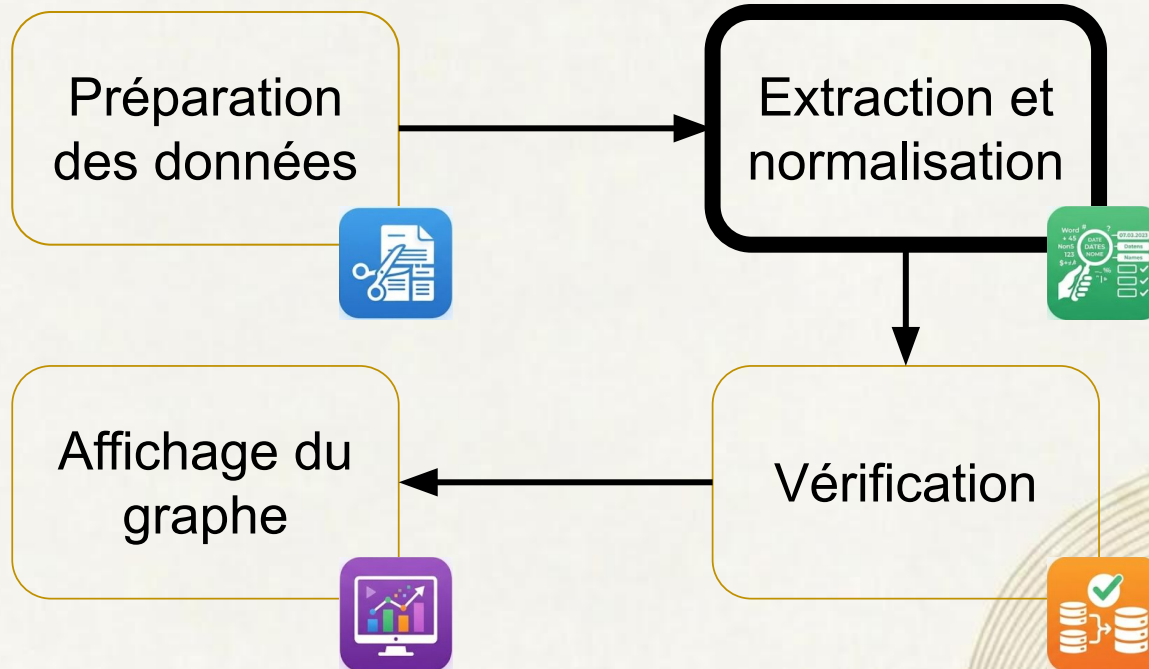
Solution trouvé : (via la bibliothèque LangChain)

- Découper du texte en segments (*RecursiveCharacterTextSplitter*)
- Chevauchement des blocs



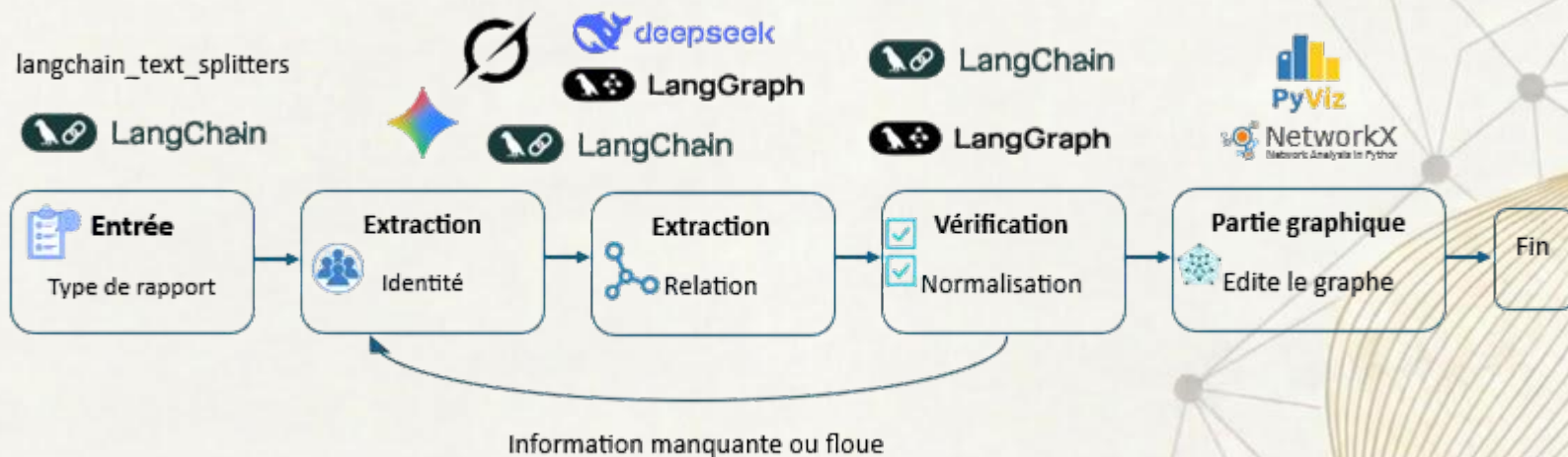
Développement

La partie de développement se découpe en **4 grandes étapes** :



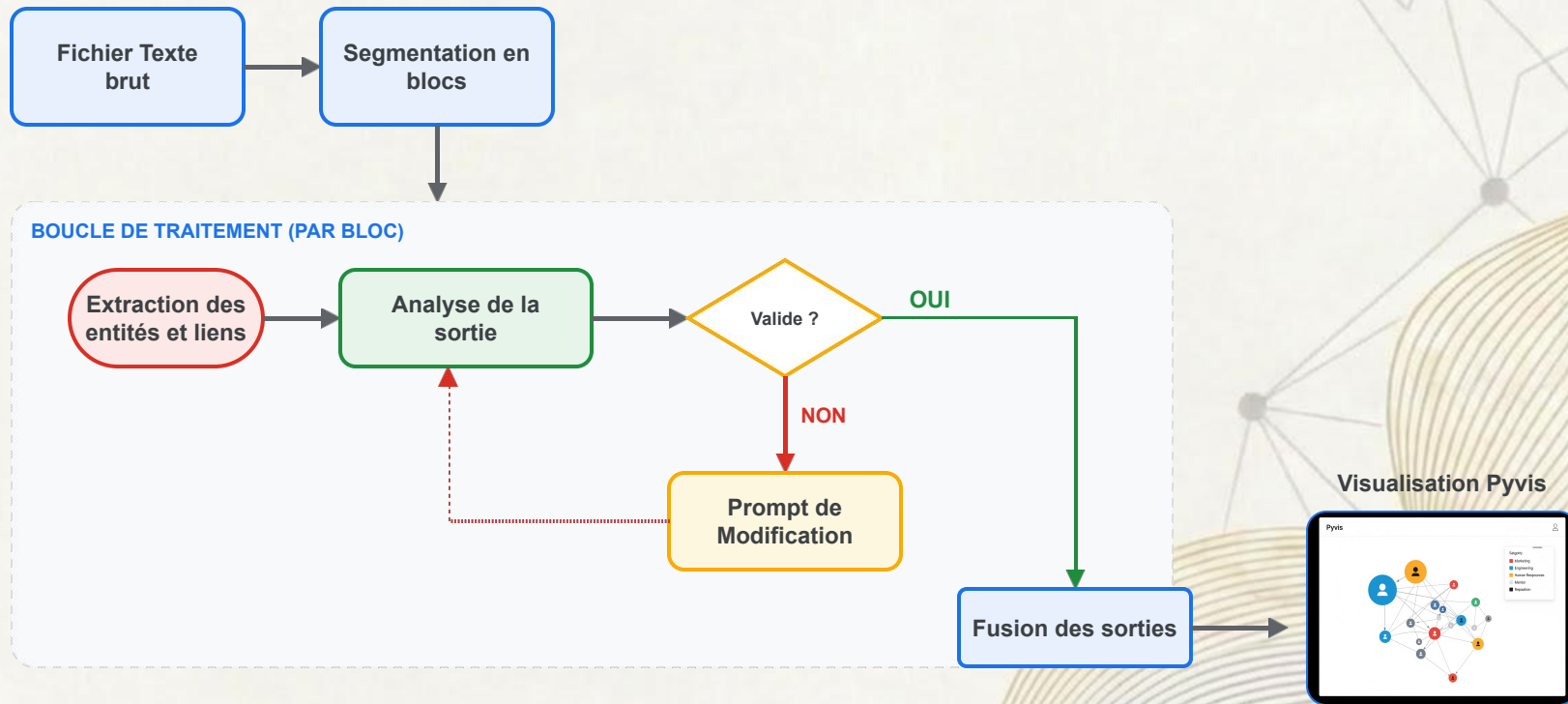


Fonctionnement de l'Algorithme



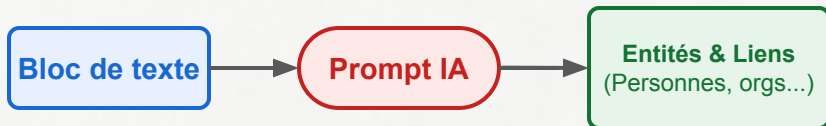


Fonctionnement de l'Algorithme





Fonctionnement de l'Algorithme d'Extraction



- 1. Entrée :** Bloc prédécoupé.
- 2. Traitement :** Application d'un prompt structuré pour guider l'extraction par l'IA.
- 3. Sortie :** Identification précise des segments de texte contenant des personnes et leurs relations mutuelles.



Exemple de traitement : Cas Enron

Utilisation de modèles variés :

- Ollama, DeepSeek, Mistral & Gemma pour l'exécution locale.
- Gemini (Google AI Studio) et ChatGPT (openAI) via API.

La méthode du chunking (page par page) est appliquée pour respecter les limites de tokens et avoir des sorties plus précises.



Normalisation de la sortie

Le Défi : L'instabilité de l'IA

L'IA peut générer :

- des hallucinations
- des formats incohérents

Risques :

- Doublons et instabilité
- Graphe de relations compromis





Normalisation de la sortie

La Solution : Programmation Objet (Pydantic)

1. Modèles Stricts

Impose le format via **BaseModel** et **Field** pour guider précisément l'IA.

```
class PersonneNormalisee(BaseModel):  
    name:str=Field(description="Nom complet, format : Nom Prénom")
```

2. Filtres Déterministes

Suppression automatique des doublons avant l'injection des données.

```
Link: Annotated[list[Lien],  
Field(description="..."),  
AfterValidator(supprimer_doublons_liens)]
```

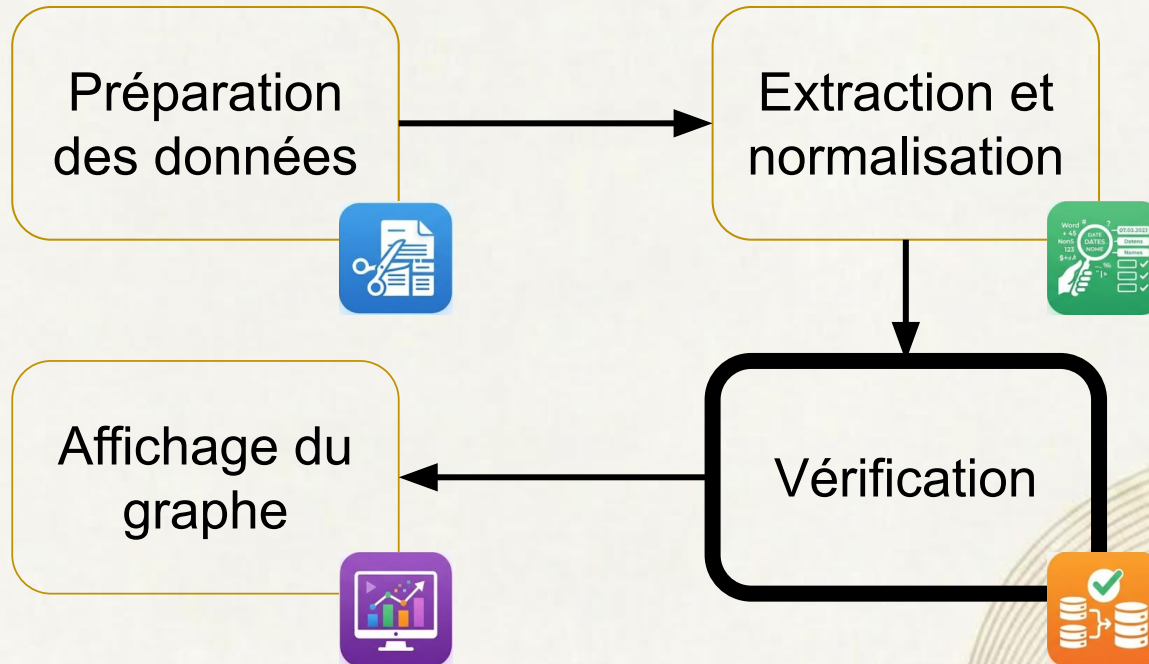
3. Fusion & Intégrité

Garantit la réciprocité des liens (A en lien avec B => B en lien avec A) et fusionne les informations.



Développement

La partie de développement se découpe en **4 grandes étapes** :





Vérification

Sécuriser l'extraction face aux incohérences de l'IA

Le Problème

Les Hallucinations

Création de liens et d'entités inexistantes, générant des incohérences critiques.

L'Approche Écartée

Comparaison manuelle : trop coûteuse et incertaine.



Nœud de Vérification

Fiabilité 100%

Contrôle automatisé de la structure, de l'ancrage et des relations.



Vérification

Garantir un résultat 100% fiable

STRUCTURE

1. Validation de la structure

Vérifie la conformité du résultat au schéma attendu.

ANCRAGE

2. Ancrage des entités

Vérifie que les noms sont présents mot pour mot dans le texte.

RELATIONS

3. Validation des liens

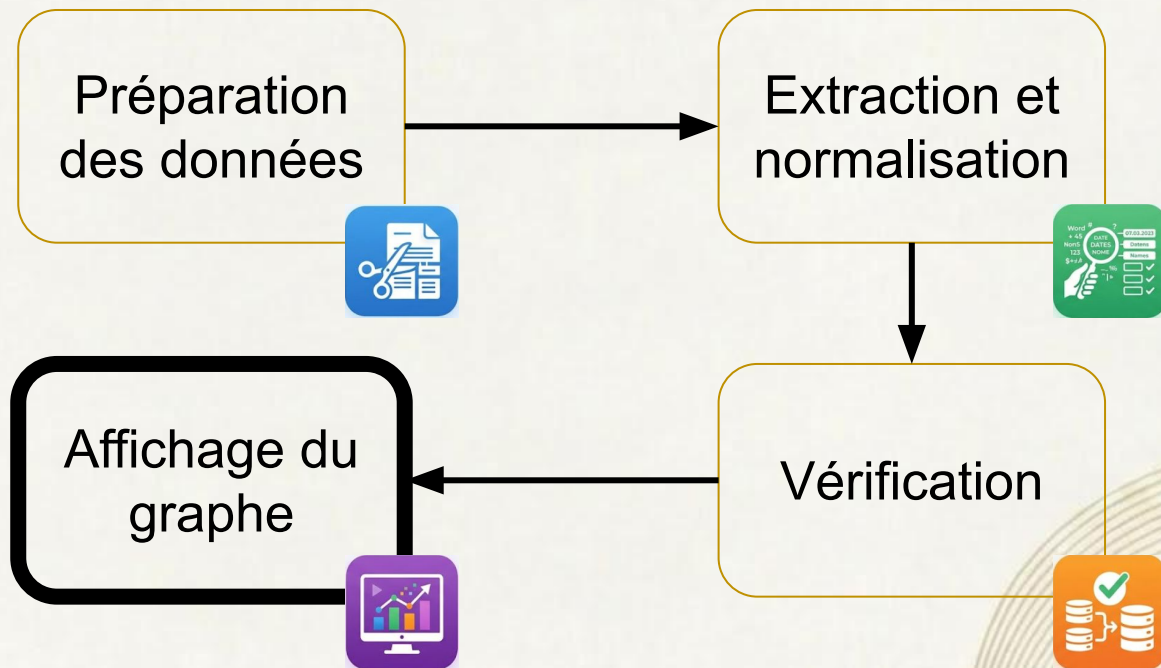
Confirme la coexistence des entités dans un même paragraphe.





Développement

La partie de développement se découpe en **4 grandes étapes** :





Affichage

Bibliothèque : Pyvis

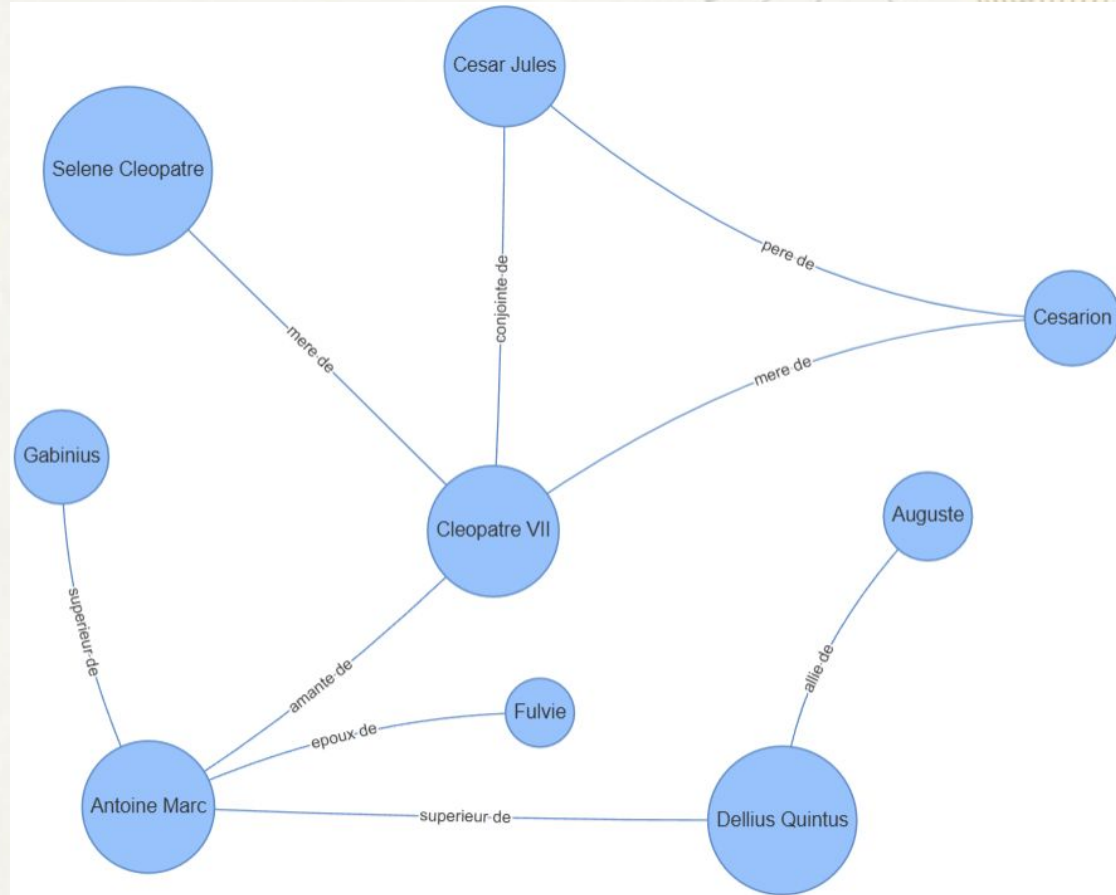
- Création de graphe en page HTML interactive

Points positifs :

- Sommets visibles
- Liens explicites

Améliorations possibles :

- Ajouter des couleurs
- Lier la taille des sommets à une logique







Affichage

ÉTAPE 1

Analyse du point d'entrée

Lecture du fichier markdown.

Comptage du nombre d'apparitions de chaque entité détectée dans le JSON.



```
▼ entities:
  ▼ 0:
    id:      "enron"
    type:    "company"
    name:    "Enron"
  ▼ 1:
    id:      "gary_mulgrew"
    type:    "person"
    name:    "Gary Mulgrew"
  ▼ 2:
    id:      "neil_coulbeck"
    type:    "person"
    name:    "Neil Coulbeck"
▼ links:
  ▼ 0:
    source:  "kenneth_lay"
    target:  "enron"
    relation: "fondateur et PDG"
    gravite: 5
  ▼ 1:
    source:  "jeffrey_skilling"
    target:  "enron"
    relation: "directeur general"
    gravite: 5
  ▼ 2:
    source:  "arthur_andersen"
    target:  "enron"
    relation: "auditeur"
    gravite: 5
```




Affichage

ÉTAPE 2

Calcul de l'échelle

Nombre de paliers (k) :

Défini par la règle de Sturges :

$$k = 1 + \log_2(n)$$

Amplitude (A) :

Écart entre les "seconds" extrêmes :

$$A = (\text{maxi} - \text{mini})/k$$

Attribution (P) :

Calcul du palier final par entité :

$$P = \text{floor}((\text{occ} - \text{mini})/A) + 1$$

La fonction retourne une **liste contenant le nom** et le palier de chaque sommet

ÉTAPE 3

Création du Graphe

Génération d'un graphe interactif Pyvis.

Création et insertion de la légende.





Sommaire

- Contexte / Objectif
- Développement
- **Mise en commun**
- Module de contrôle
- Limites et difficultés
- Méthodologie / Gantt
- Conclusion





Mise en commun

Gestion des ruptures de compatibilité et robustesse du flux

La fluidité théorique entre extraction et visualisation s'est heurtée à une réalité technique : les outils ne communiquaient pas nativement entre eux.

<> IA vs Python (JSON)

Élimination du texte conversationnels via des instructions fortes et des schémas **Pydantic** rigides pour garantir l'exploitabilité.



Boucles de LangGraph

Calibration du **State** et des arêtes conditionnelles pour corriger la donnée sans altérer l'arborescence.



Sommaire

- Contexte / Objectif
- Développement
- Mise en commun
- **Module de contrôle**
- Limites et difficultés
- Méthodologie / Gantt
- Conclusion



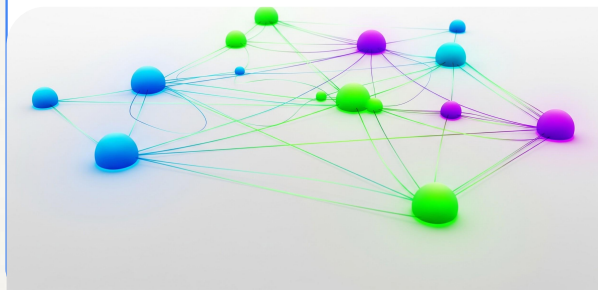


Module de contrôle

ÉTAPE 1

Génération du Graphe Parfait

Création aléatoire d'entités (personnes, entreprises) et de liens logiques (famille, affaires).



ÉTAPE 2

Rapport d'audit fictif avec "Bruit" sans relations

Conversion en texte Markdown avec injection de paragraphes hors-sujet (recettes, météo) pour piéger le LLM.





Module de contrôle

ÉTAPE 3

Intégration de bruit avec relation

But : évaluer la robustesse du modèle face à des données bruitées pouvant entraîner une confusion dans les liens .

Importer dans le fichier texte une liste contenant la légende du roi Arthur.



ÉTAPE 4

Analyse des données

Comparaison : Confrontation du graphe généré par le LLM avec notre jeu de tests de référence.

Identification des erreurs :
Mesure des taux de Faux Positifs et de Faux Négatifs

Évaluation globale : Calcul de la précision finale du graphe



ÉTAPE 5

Automatisation du test

Sauvegarde des différentes statistiques du modèle.

Calcul des moyennes des différents tests et affichage dans la console.

Automatisation des tests.





Module de contrôle

Analyse des statistiques (112 test(s))

ENTITÉS (Sommets) :

- Inventés (Faux Positifs) : 34.749 %
- Oubliés (Faux Négatifs) : 3.692 %

RELATIONS (Liens) :

- Inventées (Faux Positifs) : 39.874 %
- Oubliées (Faux Négatifs) : 22.69 %

SCORE GLOBAL :

- Précision globale du graphe : 74.751 %

43.243	2.703	48.75	24.375	70.232
29.73	8.108	49.4	25.	71.940
29.73	0.0	38.75	26.9	76.155
43.243	24.324	40.6	22.5	67.333
27.027	2.703	41.25	21.25	76.942
56.757	2.703	47.5	24.375	67.166
48.649	2.703	46.25	26.875	68.881
10.811	35.135	16.6	44.37	73.196
2.703	32.432	26.0	43.125	73.710
35.135	0.0	45.6	13.125	76.535
27.027	0.0	44.4	27.5	75.268
35.135	0.0	45.0	27.5	73.091
48.649	5.405	49.1	24.375	68.118
37.838	13.514	48.8	21.25	70.149
40.541	2.703	37.5	32.5	71.689
40.541	0.0	33.75	40.0	71.427
40.541	2.703	40.0	60.0	64.189
32.432	0.0	35.0	20.625	77.986
32.432	8.108	36.25	23.75	74.865
37.838	0.0	33.75	16.25	78.041
32.432	0.0	37.5	22.5	76.892
24.324	2.703	40.0	10.625	80.587
43.243	0.0	40.0	13.75	75.752
29.73	5.405	40.0	24.375	75.123



Module de contrôle

Démarche sur les sommets :

- Traduction du résultat sous forme de liste
- On fait la différence entre deux listes.
 $\text{manquants} = \text{set_depart} - \text{set_arriver}$
- On effectue un nouveau parcours avec les alias
- Formule qui permet d'avoir la statistique :

$\text{stat_faux_pos_s} = \text{round}(\text{len}(\text{manquants}) / \text{len_d} * 100, 3)$

Analyse :

- Concernant les faux positifs (les éléments inventés), cela s'explique peut-être par le fait que le LLM traite les différents textes comme un seul et même document divisé en plusieurs paragraphes.
- Quant aux faux négatifs (les éléments oubliés), le taux d'échec moyen global est de 3,692 %, ce qui représente moins d'un sommet manquant par test.

Résultat :

ENTITÉS (Sommets) :

- Inventés (Faux Positifs) : 54.054 %
- Oubliés (Faux Négatifs) : 0.0 %



Module de contrôle

Démarche sur les liens :

problème comment avoir une détection optimale sur les relations : 1) faire appel au LLM

2) utiliser spacy

3) utiliser Sentence Transformers

Sentence Transformers est une méthode qui nous permet d'avoir un taux de similitude entre deux phrases.

formule statistique :

```
stat_faux_pos_l = round((len(list_dep_lien) / 2 - nbre_resultat) / len_d * 100, 3)
```

Analyse :

- Concernant les oublis, cela peut s'expliquer par la dispersion de l'information liée aux liens.
- Quant aux faux négatifs (les liens non détectés), cette erreur peut être due à une trop grande dispersion de l'information dans le texte source.

Résultat :

RELATIONS (Liens) :

- Inventées (Faux Positifs) : 38.874 %
- Oubliées (Faux Négatifs) : 22.69 %



Sommaire

- Contexte / Objectif
- Développement
- Mise en commun
- Module de contrôle
- **Limites et difficultés**
- Méthodologie / Gantt
- Conclusion





Limites et difficultés



Complexité Technique

Défis liés à l'intégration des API et à la gestion des modèles de langage volumineux.



Contraintes de Temps

Délais serrés pour le développement des fonctionnalités et les phases de tests intensifs.



Qualité des Données

Difficultés rencontrées avec l'extraction et principalement les liens.



Sommaire

- Contexte / Objectif
- Développement
- Mise en commun
- Module de contrôle
- Limites et difficultés
- **Méthodologie / Gantt**
- Conclusion





Méthodes et Organisation

Utilisation de la méthode “agile”

The screenshot displays a ClickUp Kanban board with the following columns and tasks:

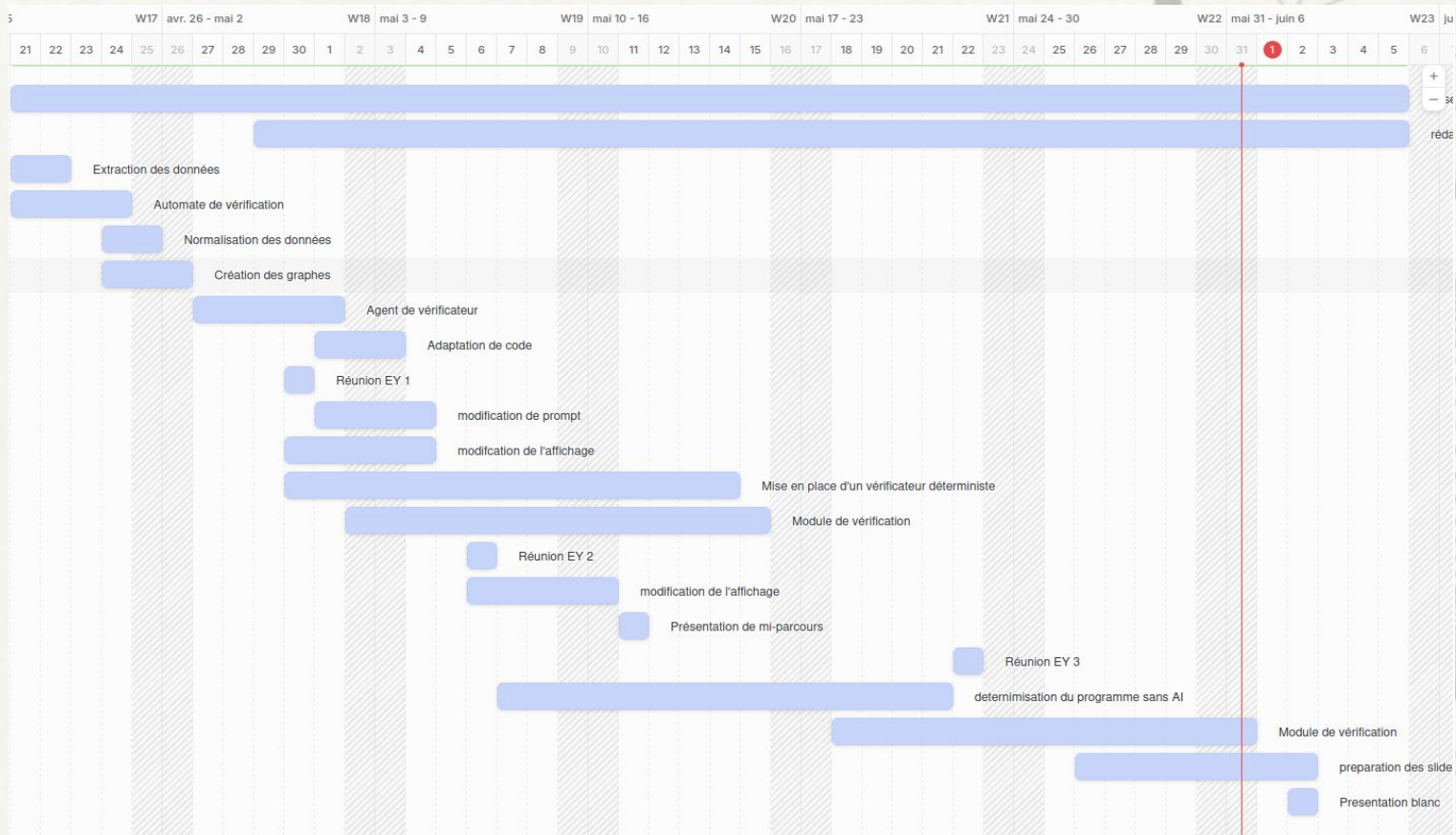
- À FAIRE (1):**
 - preparation de la soutenance (Assignee: AB, Due: mai 19 - Il y a 4 jours)
- EN COURS (2):**
 - redaction du rapport (Assignee: R, Due: avr. 29 - ven., 3 sous-tâches)
 - finalisation du module de test (Assignee: AB, Due: mai 11 - Il y a 6 jours)
- REVUE DE CODE (1):**
 - Jupyter Notebook + mettre un .py (Assignee: AB, Due: Il y a 6 jours - Aujourd'hui, 3 sous-tâches)
- ACHEVÉ (16):**
 - maï 1, 9:30 am - 5:00 pm
 - Réunion EY 2 (Assignee: AB, Due: mai 6, 10:30 am - 11:30 am)
 - modification de l'affichage (Assignee: AB, Due: mai 6 - mai 10)
 - Présentation de mi-parcours (Assignee: AB, Due: mai 11, 9:00 am - 10:00 am)
 - Prise en compte des remarque du prof (Due: mai 11)
 - determinisation du programme sans AI (Assignee: AB, Due: mai 11 - Il y a 6 jours)

+ Ajouter un groupe





Diagramme de Gantt final





Sommaire

- Contexte / Objectif
- Développement
- Mise en commun
- Module de contrôle
- Limites et difficultés
- Méthodologie / Gantt
- **Conclusion**





Résumé / Conclusion

